DWES · 2. Chuleta Spring Boot para el examen

Nov 8, 2026 · @vielsajr

0. Orden de trabajo en el examen

Todo el código de esta chuleta viene del proyecto de solución (tema: monedas), que he compilado y probado. **Para adaptarlo a otro tema, cambia `Moneda` por tu clase y `moneda` / `monedas` por tus nombres.** Los comentarios `// CAMBIAR` marcan lo que depende del tema.

1. **Proyecto** (A): Initializr con Web, Thymeleaf, Validation, Lombok y DevTools → abrir en IntelliJ → `application.properties`.
2. **Paquetes**: `model`, `service`, `controller`, `data`, `util`, `config`, todos dentro del paquete principal.
3. **Modelo** (B): clases con Lombok y validaciones.
4. **Data** (B): datos iniciales.
5. **Servicios** (B): interfaz + `Impl` con `@Service`.
6. **Layout** (F): `fragments/layout.html`. Arrancar y ver que `/` carga.
7. **Listado y detalle** (E + F). Probar.
8. **Formulario de alta y edición** con validación (E + F). Probar errores.
9. **Borrado** y mensajes flash. **Imágenes** (G).
10. **Extras**: filtro, búsqueda, inicio con últimas.

Consejo: arranca la app después de cada paso. Un error aislado se encuentra en segundos; diez errores juntos, no.

Estructura de carpetas

```
appcoleccion/
├── pom.xml
└── src/main/
    ├── java/es/daw/appcoleccion/          ← paquete principal
(Group.Artifact)
    |   ├── AppcoleccionApplication.java
    |   ├── model/       Moneda.java, Coleccion.java, Coleccionista.java
    |   ├── data/        MonedaData.java, ColeccionData.java,
ColeccionistaData.java
    |   ├── service/     MonedaService.java, MonedaServiceImpl.java, ... (x3)
    |   ├── controller/  InicioController, MonedaController,
```

```
ColeccionController, ColeccionistaController
    |   ├── util/        ImagenUtil.java
    |   └── config/      WebConfig.java
    └── resources/
        ├── application.properties
        ├── messages.properties            ← opcional (mensajes de
conversión)
        ├── static/css/estilos.css
        ├── static/imagenes/               ← imágenes de los datos iniciales
        └── templates/
            ├── fragments/layout.html
            ├── index.html
            ├── monedas/      lista.html, detalle.html, formulario.html
            ├── colecciones/  lista.html
            └── coleccionistas/ lista.html, detalle.html
```

Mapa de rutas

Método	URL	Hace	Vista / destino
GET	`/`	Inicio con las últimas incorporadas	`index`
GET	`/monedas?coleccionId=&nombre=`	Listado + filtro + búsqueda	`monedas/lista`
GET	`/monedas/{id}`	Detalle	`monedas/detalle`
GET	`/monedas/nueva`	Formulario vacío	`monedas/formulario`
GET	`/monedas/{id}/editar`	Formulario relleno	`monedas/formulario`
POST	`/monedas/guardar`	Alta o edición	`redirect:/monedas/{id}`
POST	`/monedas/{id}/borrar`	Borrado	`redirect:/monedas`
GET	`/colecciones`	Listado	`colecciones/lista`
GET	`/coleccionistas` y `/coleccionistas/{id}`	Listado y detalle	`coleccionistas/...`

A. Crear el proyecto

A.1 Spring Initializr (start.spring.io o IntelliJ → New Project → Spring Boot)

Campo	Valor
Project	**Maven**
Language	Java
Spring Boot	La más reciente estable (sin SNAPSHOT ni M/RC)
Group	`es.daw` (o el que diga el profesor)
Artifact	`appcoleccion`
Packaging	Jar
Java	La de tu JDK (17 o 21)

Dependencia (nombre en Initializr)	Para qué	¿En el ejercicio?
Spring Web	Controladores, Tomcat embebido	Sí
Thymeleaf	Plantillas HTML	Sí
Validation	`@Valid`, `@NotBlank`, `@Min`…	Sí (se olvida mucho)
Lombok	`@Data`, constructores	Sí
Spring Boot DevTools	Reinicio automático	Sí (cómodo)
Spring Data JPA + MySQL Driver	Base de datos	**No** (siguiente unidad)
Spring Security	Login	**No**: bloquea todas las páginas con un login

Generate → descomprimir → IntelliJ: Open → carpeta. Espera a que Maven descargue (barra inferior).

¿Te olvidaste una dependencia? Añádela al `pom.xml` dentro de `<dependencies>` y pulsa el botón de recargar Maven (la «m» con flechas):

```
<!-- pom.xml -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-validation</artifactId>   <!-- CAMBIAR: -
web, -thymeleaf, -validation -->
</dependency>
```

Con Spring Boot 4, Initializr puede escribir `spring-boot-starter-webmvc` en lugar de `spring-boot-starter-web`; sirven para lo mismo. No pongas versión en los starters: la hereda del `<parent>`.

Lombok en IntelliJ: si los getters salen en rojo, instala el plugin Lombok y activa *Settings → Build → Compiler → Annotation Processors → Enable annotation processing*.

A.2 application.properties

Archivo `src/main/resources/application.properties`:

```
spring.application.name=appcoleccion
server.port=8080

# Ver cambios en las plantillas sin reiniciar
spring.thymeleaf.cache=false

# Subida de imágenes
spring.servlet.multipart.max-file-size=5MB
spring.servlet.multipart.max-request-size=5MB
# Carpeta EXTERNA al JAR, relativa a donde se arranca la app (CAMBIAR si
quieres)
file.upload-dir=uploads
```

Nunca pongas un comentario al final de una línea (`file.upload-dir=uploads # carpeta`): en `.properties` eso forma parte del valor.

Opcional, `src/main/resources/messages.properties` (mensajes cuando el usuario escribe algo no convertible):

```
typeMismatch.java.lang.Integer=Debe ser un número entero
typeMismatch.java.lang.Double=Debe ser un número (usa punto para los
decimales)
typeMismatch.java.time.LocalDate=Fecha no válida
```

A.3 Arrancar

Botón ▶ sobre la clase `...Application`, o `mvn spring-boot:run`. Correcto si ves `Tomcat started on port 8080`. Abre http://localhost:8080/.

B. Las capas, archivo por archivo

B.1 Clase principal — `AppcoleccionApplication.java` (raíz del paquete)

La genera Initializr. No se toca. **Todo lo demás debe estar en subpaquetes de este.**

```
@SpringBootApplication
public class AppcoleccionApplication {
    public static void main(String[] args) {
        SpringApplication.run(AppcoleccionApplication.class, args);
    }
}
```

B.2 Modelo — `model/Moneda.java`

Plantilla de «pieza» con las tres validaciones exigidas (obligatorio, rango numérico, fecha) y la relación con `Coleccion`.

```
package es.daw.appcoleccion.model;

import jakarta.validation.constraints.*;          // NotBlank, Size, NotNull,
Min, Max, PositiveOrZero, PastOrPresent
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;
import org.springframework.format.annotation.DateTimeFormat;
import java.time.LocalDate;

@Data                    // getters, setters, toString, equals...
@NoArgsConstructor       // OBLIGATORIO para formularios
@AllArgsConstructor      // para crear los datos iniciales en una línea
public class Moneda {    // CAMBIAR nombre de la clase

    private Long id;     // Long (objeto): en un alta llega null

    @NotBlank(message = "El nombre es obligatorio")                // ←
OBLIGATORIO
    @Size(max = 80, message = "El nombre no puede superar 80 caracteres")
    private String nombre;
```

```
    @Size(max = 500, message = "La descripción no puede superar 500
caracteres")
    private String descripcion;

    @NotNull(message = "El año es obligatorio")
    @Min(value = 1, message = "El año debe ser mayor que 0")          // ←
RANGO NUMÉRICO
    @Max(value = 2100, message = "El año no puede ser mayor que 2100")
    private Integer anio;

    @NotNull(message = "El valor estimado es obligatorio")
    @PositiveOrZero(message = "El valor no puede ser negativo")
    private Double valorEstimado;

    @NotNull(message = "La fecha de incorporación es obligatoria")
    @PastOrPresent(message = "La fecha de incorporación no puede ser futura")
// ← FECHA
    @DateTimeFormat(iso = DateTimeFormat.ISO.DATE)   // <input type="date">
envía yyyy-MM-dd
    private LocalDate fechaIncorporacion;

    private String imagen;          // solo el NOMBRE del fichero

    @NotBlank(message = "El país es obligatorio")       // CAMBIAR: 2
atributos propios del tema
    private String pais;

    @NotBlank(message = "El material es obligatorio")
    private String material;

    private Coleccion coleccion;    // Moneda TIENE una Coleccion (la
Coleccion no conoce sus monedas)
}
```

`model/Coleccion.java` y `model/Coleccionista.java` siguen el mismo patrón, más cortos:

```
@Data @NoArgsConstructor @AllArgsConstructor
public class Coleccion {
    private Long id;
    @NotBlank(message = "El nombre es obligatorio")
    private String nombre;
    private String descripcion;
}

@Data @NoArgsConstructor @AllArgsConstructor
```

```
public class Coleccionista {
    private Long id;
    @NotBlank(message = "El nombre es obligatorio")
    private String nombre;
    @NotBlank(message = "El email es obligatorio")
    @Email(message = "El email no es válido")
    private String email;
    @NotNull(message = "La fecha de alta es obligatoria")
    @PastOrPresent(message = "La fecha de alta no puede ser futura")
    @DateTimeFormat(iso = DateTimeFormat.ISO.DATE)
    private LocalDate fechaAlta;
}
```

B.3 Datos iniciales — `data/ColeccionData.java` y `data/MonedaData.java`

Clases sin anotaciones, con un método `static` que devuelve una lista **modificable** (`new ArrayList<>()` , nunca `List.of`). El enunciado pide mínimo 3 colecciones, 10 piezas y 4 coleccionistas.

```
public class ColeccionData {
    private ColeccionData() { }     // no se crean objetos de esta clase

    public static List<Coleccion> getColecciones() {
        List<Coleccion> colecciones = new ArrayList<>();
        colecciones.add(new Coleccion(1L, "Euro", "Monedas de la zona
euro"));       // CAMBIAR datos
        colecciones.add(new Coleccion(2L, "Peseta", "Monedas españolas
anteriores al euro"));
        colecciones.add(new Coleccion(3L, "Mundo", "Monedas de otros
países"));
        return colecciones;
    }
}

public class MonedaData {
    private MonedaData() { }

    // Recibe las colecciones para que cada Moneda apunte al MISMO objeto que
tiene el servicio
    public static List<Moneda> getMonedas(List<Coleccion> colecciones) {
        Coleccion euro = buscar(colecciones, 1L);
        Coleccion peseta = buscar(colecciones, 2L);

        List<Moneda> monedas = new ArrayList<>();
        // orden = orden de los atributos en la clase (lo exige
@AllArgsConstructor)
```

```
        monedas.add(new Moneda(1L, "2 euros España", "Primera serie", 1999,
2.5,
                LocalDate.of(2024, 1, 15), "2euros-espana.png", "España",
"Bimetálica", euro));
        monedas.add(new Moneda(5L, "100 pesetas", "Aluminio-bronce", 1982,
1.0,
                LocalDate.of(2023, 11, 10), "100pesetas-1982.png", "España",
"Aluminio-bronce", peseta));
        // ... hasta 10
        return monedas;
    }

    private static Coleccion buscar(List<Coleccion> colecciones, Long id) {
        return colecciones.stream().filter(c ->
c.getId().equals(id)).findFirst().orElse(null);
    }
}
```

B.4 Servicio — `service/MonedaService.java` (interfaz)

```
public interface MonedaService {
    List<Moneda> listarTodas();
    Optional<Moneda> buscarPorId(Long id);
    Moneda guardar(Moneda moneda);                          // alta (id null)
o edición
    boolean eliminar(Long id);
    List<Moneda> buscar(Long coleccionId, String nombre);   // filtro +
búsqueda
    List<Moneda> ultimasIncorporadas(int cantidad);          // inicio
}
```

B.5 Implementación en memoria — `service/MonedaServiceImpl.java`

```
@Service                                                // ← aquí, NO en la
interfaz
public class MonedaServiceImpl implements MonedaService {

    private final List<Moneda> monedas;
    private Long siguienteId;

    public MonedaServiceImpl(ColeccionService coleccionService) {   // Spring
inyecta el otro servicio
        this.monedas = MonedaData.getMonedas(coleccionService.listarTodas());
        this.siguienteId =
monedas.stream().mapToLong(Moneda::getId).max().orElse(0L) + 1;
```

```
    }

    @Override
    public List<Moneda> listarTodas() {
        return
monedas.stream().sorted(Comparator.comparing(Moneda::getNombre)).toList();
    }

    @Override
    public Optional<Moneda> buscarPorId(Long id) {
        return monedas.stream().filter(m ->
m.getId().equals(id)).findFirst();
    }

    @Override
    public Moneda guardar(Moneda moneda) {
        if (moneda.getId() == null) {                // ALTA
            moneda.setId(siguienteId++);
            monedas.add(moneda);
        } else {                                     // EDICIÓN: sustituir
el que tenga ese id
            for (int i = 0; i < monedas.size(); i++) {
                if (monedas.get(i).getId().equals(moneda.getId())) {
                    monedas.set(i, moneda);
                    return moneda;
                }
            }
            monedas.add(moneda);
        }
        return moneda;
    }

    @Override
    public boolean eliminar(Long id) {
        return monedas.removeIf(m -> m.getId().equals(id));   // true si
borró algo
    }

    @Override
    public List<Moneda> buscar(Long coleccionId, String nombre) {
        return monedas.stream()
                .filter(m -> coleccionId == null
                        || (m.getColeccion() != null &&
m.getColeccion().getId().equals(coleccionId)))
                .filter(m -> nombre == null || nombre.isBlank()
                        ||
```

```
m.getNombre().toLowerCase().contains(nombre.trim().toLowerCase()))
                .sorted(Comparator.comparing(Moneda::getNombre))
                .toList();
    }

    @Override
    public List<Moneda> ultimasIncorporadas(int cantidad) {
        return monedas.stream()

.sorted(Comparator.comparing(Moneda::getFechaIncorporacion).reversed())
                .limit(cantidad)
                .toList();
    }
}
```

Servicios sencillos (`ColeccionService` , `ColeccionistaService`) solo necesitan `listarTodas()` y `buscarPorId()` , con la lista cargada en el constructor: `this.colecciones = ColeccionData.getColecciones();` . En `buscarPorId` de colecciones, devuelve `Optional.empty()` si el id es null (llega null cuando el `<select>` está sin elegir).

C. Java de bolsillo

```
// --- Clase a mano (sin Lombok) ---
public class Producto {
    private String nombre;                        // atributo privado
    public Producto() { }                         // constructor vacío
    public Producto(String nombre) { this.nombre = nombre; }
    public String getNombre() { return nombre; } // getter (boolean →
isActivo())
    public void setNombre(String nombre) { this.nombre = nombre; }
    @Override public String toString() { return nombre; }
}

// --- Interfaz + implementación + herencia ---
public interface Servicio { List<X> listar(); }
public class ServicioImpl implements Servicio { @Override public List<X>
listar() { ... } }
public class Perro extends Animal { public Perro(String n) { super(n); } }

// --- Listas (modificables) ---
List<Moneda> lista = new ArrayList<>();
lista.add(m);  lista.get(0);  lista.size();  lista.isEmpty();
lista.set(i, m);                                  // sustituir (editar)
```

```
lista.removeIf(x -> x.getId().equals(id));         // borrar por condición
for (Moneda x : lista) { ... }
for (int i = 0; i < lista.size(); i++) { ... }

// --- Comparar ---
texto.equals("hola")            texto.equalsIgnoreCase("HOLA")
id1.equals(id2)                 // Long/Integer: NUNCA ==
miEnum == Dificultad.MEDIA      // enum: sí ==

// --- Strings útiles ---
texto.toLowerCase().contains(busqueda.toLowerCase())   // búsqueda sin
mayúsculas
texto.isBlank()   texto.trim()   texto.startsWith("A")   texto.length()

// --- Streams ---
lista.stream().filter(m -> m.getAnio() > 2000).toList();
// filtrar
lista.stream().map(Moneda::getNombre).toList();
// transformar
lista.stream().sorted(Comparator.comparing(Moneda::getNombre)).toList();
// ordenar A-Z
lista.stream().sorted(Comparator.comparing(Moneda::getValorEstimado).reversed
()).toList(); // Z-A
lista.stream().limit(5).toList();
// primeros 5
lista.stream().filter(...).count();
// contar (long)
lista.stream().anyMatch(m -> m.getEmail().equalsIgnoreCase(email));
// ¿existe?
lista.stream().mapToDouble(Moneda::getValorEstimado).sum();
// sumar
lista.stream().mapToDouble(Moneda::getValorEstimado).average().orElse(0);
// media
lista.stream().max(Comparator.comparing(Moneda::getValorEstimado));
// Optional del máximo
lista.stream().mapToLong(Moneda::getId).max().orElse(0L) + 1;
// siguiente id

// --- Optional ---
Optional<Moneda> o = lista.stream().filter(m ->
m.getId().equals(id)).findFirst();
o.isPresent()  o.isEmpty()  o.get()  o.orElse(null)  o.orElseThrow()
o.ifPresent(m -> ...)

// --- Fechas (java.time) ---
LocalDate.of(2024, 3, 20)   LocalDate.now()   fecha.isBefore(otra)
```

```
fecha.getYear()

// --- Ternario ---
String msg = esAlta ? "Creada" : "Modificada";

// --- try/catch (obligatorio con ficheros) ---
try { Files.write(ruta, bytes); } catch (IOException e) { /* gestionar */ }
```

Imports que IntelliJ a veces no adivina: `java.util.List`, `java.util.ArrayList`, `java.util.Optional`, `java.util.Comparator`, `java.time.LocalDate`, `org.springframework.ui.Model` (no el de otro paquete), `jakarta.validation.Valid`, `jakarta.validation.constraints.*`. Atajo de IntelliJ: Alt+Enter sobre la palabra en rojo.

D. Anotaciones

Anotación	Dónde va	Qué hace
`@SpringBootApplication`	Clase principal	Arranca Spring y escanea los subpaquetes
`@Controller`	Clase en `controller/`	Controlador que devuelve **vistas**
`@RestController`	Clase	Devuelve **datos** (JSON/texto). No para Thymeleaf
`@RequestMapping("/monedas")`	Clase	Prefijo de todas las rutas del controlador
`@GetMapping("/{id}")`	Método	Atiende GET
`@PostMapping("/guardar")`	Método	Atiende POST (formularios)
`@PutMapping`, `@DeleteMapping`	Método	PUT / DELETE (APIs; los forms HTML no los envían)
`@PathVariable Long id`	Parámetro	Lee `{id}` de la ruta
`@RequestParam(required = false) String nombre`	Parámetro	Lee `?nombre=...`
`@RequestBody`	Parámetro	Lee JSON del cuerpo (APIs REST; **no** en formularios Thymeleaf)

Anotación	Dónde va	Qué hace
`@ModelAttribute("moneda") Moneda m`	Parámetro	Rellena un objeto con los campos del formulario
`@ModelAttribute("colecciones")`	Método	Añade ese dato al Model de todas las vistas del controlador
`@Valid`	Delante del `@ModelAttribute`	Valida con las anotaciones del modelo
`@Service`	Clase `...ServiceImpl`	Bean de lógica de negocio
`@Repository`	Interfaz/clase de datos	Bean de acceso a datos (JPA)
`@Component`	Clase	Bean genérico
`@Configuration`	Clase en `config/`	Clase de configuración
`@Autowired`	Constructor, atributo o setter	Inyecta un bean (opcional con un solo constructor)
`@Value("${file.upload-dir}")`	Atributo de un bean	Inyecta una propiedad
`@Override`	Método	Sobrescribe/implementa (Java)
`@NotBlank` `@NotNull` `@Size` `@Min` `@Max` `@Email` `@Pattern` `@Past` `@PastOrPresent` `@Future` `@Positive` `@PositiveOrZero` `@DecimalMin`	Atributos del modelo	Validación
`@DateTimeFormat(iso = ISO.DATE)`	Atributo `LocalDate`	Convierte el texto del input date
`@Data` `@NoArgsConstructor` `@AllArgsConstructor`	Clase del modelo	Lombok

Las que se confunden

Esto	vs. esto	Regla
`@Controller`	`@RestController`	Vista HTML → `@Controller`. JSON → `@RestController`
`@PathVariable`	`@RequestParam`	Identifica UNO (`/monedas/5`) vs. filtra/busca (`?nombre=x`)
`@RequestParam`	`@ModelAttribute`	Un campo suelto vs. el objeto entero del formulario
`@ModelAttribute`	`@RequestBody`	Formulario HTML vs. JSON (API)
`@NotNull`	`@NotEmpty` / `@NotBlank`	No null (números, fechas, objetos) vs. texto no vacío. Para `String` usa `@NotBlank`
`@Min` / `@Max`	`@Size`	Valor de un número vs. longitud de un texto
`@Past`	`@PastOrPresent`	Estrictamente antes de hoy vs. hoy incluido (fecha de alta → `@PastOrPresent`)
`th:insert`	`th:replace`	Mete dentro del contenedor vs. lo sustituye
`${...}`	`*{...}`	Variable del Model vs. campo del `th:object`
`redirect:`	`forward:`	Nueva petición (URL cambia, Model se pierde) vs. interna
`model.addAttribute`	`redirectAttrs.addFlashAttribute`	Para la vista que devuelves vs. para la página tras un redirect

Esto	vs. esto	Regla
`return "monedas/lista"`	`return "redirect:/monedas"`	Nombre de PLANTILLA (sin `/` inicial) vs. URL (con `/`)
`@Service` en la interfaz	en la `Impl`	Siempre en la implementación

E. Controlador CRUD completo — `controller/MonedaController.java`

Es el archivo más importante del examen. Cada método está numerado para buscarlo rápido.

```
package es.daw.appcoleccion.controller;

// imports: Coleccion, Moneda, ColeccionService, MonedaService, ImagenUtil,
jakarta.validation.Valid,
// org.springframework.beans.factory.annotation.Value, stereotype.Controller,
ui.Model,
// validation.BindingResult, web.bind.annotation.*,
web.multipart.MultipartFile,
// web.servlet.mvc.support.RedirectAttributes, java.io.IOException,
java.util.List, java.util.Optional

@Controller
@RequestMapping("/monedas")                          // CAMBIAR
public class MonedaController {

    private final MonedaService monedaService;        // depende de la
INTERFAZ
    private final ColeccionService coleccionService;

    @Value("${file.upload-dir}")
    private String uploadDir;

    public MonedaController(MonedaService monedaService, ColeccionService
coleccionService) {
        this.monedaService = monedaService;
        this.coleccionService = coleccionService;
    }

    // 0. Disponible en TODAS las vistas de este controlador (select del form
y filtro)
```

```
    @ModelAttribute("colecciones")
    public List<Coleccion> cargarColecciones() {
        return coleccionService.listarTodas();
    }

    // 1. LISTAR + FILTRAR + BUSCAR     GET /monedas?
coleccionId=2&nombre=euro
    @GetMapping
    public String listar(@RequestParam(required = false) Long coleccionId,
                         @RequestParam(required = false) String nombre,
                         Model model) {
        model.addAttribute("monedas", monedaService.buscar(coleccionId,
nombre));
        model.addAttribute("coleccionId", coleccionId);   // para dejar
marcado el filtro
        model.addAttribute("nombre", nombre);
        return "monedas/lista";
    }

    // 2. DETALLE     GET /monedas/5
    @GetMapping("/{id}")
    public String detalle(@PathVariable Long id, Model model,
RedirectAttributes redirectAttrs) {
        Optional<Moneda> moneda = monedaService.buscarPorId(id);
        if (moneda.isEmpty()) {                                    // GESTIÓN
DE "NO EXISTE"
            redirectAttrs.addFlashAttribute("error", "No existe la moneda " +
id);
            return "redirect:/monedas";
        }
        model.addAttribute("moneda", moneda.get());
        return "monedas/detalle";
    }

    // 3. FORMULARIO DE ALTA     GET /monedas/nueva
    @GetMapping("/nueva")
    public String nueva(Model model) {
        Moneda moneda = new Moneda();
        moneda.setColeccion(new Coleccion());    // para que *{coleccion.id}
no sea null
        model.addAttribute("moneda", moneda);
        model.addAttribute("titulo", "Nueva moneda");
        return "monedas/formulario";
    }

    // 4. FORMULARIO DE EDICIÓN     GET /monedas/5/editar   (mismo
```

```
formulario, objeto relleno)
    @GetMapping("/{id}/editar")
    public String editar(@PathVariable Long id, Model model,
RedirectAttributes redirectAttrs) {
        Optional<Moneda> moneda = monedaService.buscarPorId(id);
        if (moneda.isEmpty()) {
            redirectAttrs.addFlashAttribute("error", "No existe la moneda " +
id);
            return "redirect:/monedas";
        }
        model.addAttribute("moneda", moneda.get());
        model.addAttribute("titulo", "Editar moneda");
        return "monedas/formulario";
    }

    // 5. GUARDAR (alta y edición)     POST /monedas/guardar
    @PostMapping("/guardar")
    public String guardar(@Valid @ModelAttribute("moneda") Moneda moneda,
                          BindingResult resultado,                 // JUSTO
detrás
                          @RequestParam(value = "archivoImagen", required =
false) MultipartFile archivo,
                          Model model,
                          RedirectAttributes redirectAttrs) {

        // 5a. La colección llega solo con el id (del <select>): buscar el
objeto completo
        Long idColeccion = moneda.getColeccion() != null ?
moneda.getColeccion().getId() : null;
        Optional<Coleccion> coleccion =
coleccionService.buscarPorId(idColeccion);
        if (coleccion.isEmpty()) {
            resultado.rejectValue("coleccion", "coleccion.obligatoria",
"Debes elegir una colección");
        } else {
            moneda.setColeccion(coleccion.get());
        }

        // 5b. Imagen: solo si se ha subido un fichero nuevo (si no, se
conserva la del campo oculto)
        if (archivo != null && !archivo.isEmpty()) {
            try {
                moneda.setImagen(ImagenUtil.guardar(archivo, uploadDir));
            } catch (IllegalArgumentException | IOException e) {
                resultado.rejectValue("imagen", "imagen.error", "No se pudo
guardar la imagen: " + e.getMessage());
```

```
            }
        }

        // 5c. Con errores: volver al formulario (SIN redirect, o se pierden
los errores)
        if (resultado.hasErrors()) {
            model.addAttribute("titulo", moneda.getId() == null ? "Nueva
moneda" : "Editar moneda");
            return "monedas/formulario";
        }

        if (moneda.getImagen() == null || moneda.getImagen().isBlank()) {
            moneda.setImagen("sin-imagen.png");
        }

        // 5d. Guardar + mensaje flash + redirect
        boolean esAlta = moneda.getId() == null;          // ANTES de guardar
(después ya tiene id)
        monedaService.guardar(moneda);
        redirectAttrs.addFlashAttribute("mensaje",
                esAlta ? "Moneda creada correctamente" : "Moneda modificada
correctamente");
        return "redirect:/monedas/" + moneda.getId();
    }

    // 6. BORRAR     POST /monedas/5/borrar   (un form HTML solo envía GET o
POST)
    @PostMapping("/{id}/borrar")
    public String borrar(@PathVariable Long id, RedirectAttributes
redirectAttrs) {
        if (monedaService.eliminar(id)) {
            redirectAttrs.addFlashAttribute("mensaje", "Moneda eliminada
correctamente");
        } else {
            redirectAttrs.addFlashAttribute("error", "No existe la moneda " +
id);
        }
        return "redirect:/monedas";
    }
}
```

E.1 Controladores sencillos (solo listado, o listado + detalle)

```
@Controller
@RequestMapping("/coleccionistas")
public class ColeccionistaController {
    private final ColeccionistaService coleccionistaService;

    public ColeccionistaController(ColeccionistaService coleccionistaService)
{
        this.coleccionistaService = coleccionistaService;
    }

    @GetMapping
    public String listar(Model model) {
        model.addAttribute("coleccionistas",
coleccionistaService.listarTodos());
        return "coleccionistas/lista";
    }

    @GetMapping("/{id}")
    public String detalle(@PathVariable Long id, Model model,
RedirectAttributes redirectAttrs) {
        Optional<Coleccionista> c = coleccionistaService.buscarPorId(id);
        if (c.isEmpty()) {
            redirectAttrs.addFlashAttribute("error", "No existe el
coleccionista " + id);
            return "redirect:/coleccionistas";
        }
        model.addAttribute("coleccionista", c.get());
        return "coleccionistas/detalle";
    }
}

// controller/InicioController.java — página de inicio con las últimas
incorporadas
@Controller
public class InicioController {
    private final MonedaService monedaService;
    public InicioController(MonedaService monedaService) { this.monedaService
= monedaService; }

    @GetMapping("/")
    public String inicio(Model model) {
```

```
        model.addAttribute("ultimas", monedaService.ultimasIncorporadas(4));
        return "index";
    }
}
```

E.2 Errores y excepciones: tres niveles

Situación	Cómo se gestiona	Dónde
Dato de formulario incorrecto	Anotación en el modelo + `@Valid` + `BindingResult`	Modelo y POST
Regla que no cabe en una anotación (email repetido, colección vacía, dos campos combinados)	`resultado.rejectValue("campo", "codigo", "mensaje")` antes de `hasErrors()`	POST
Id que no existe	`Optional.isEmpty()` → flash `error` + redirect al listado	GET detalle/editar, POST borrar

Alternativa para el id inexistente [complementario]: lanzar `throw new ResponseStatusException(HttpStatus.NOT_FOUND, "No existe");`, que muestra la página de error 404 de Spring. Y para capturar excepciones de todos los controladores, una clase `@ControllerAdvice` con métodos `@ExceptionHandler(MiExcepcion.class)`.

F. Plantillas Thymeleaf

Todas en `src/main/resources/templates/`. El nombre que devuelve el controlador es la ruta sin `.html`.

F.1 Layout común — `templates/fragments/layout.html`

Un solo archivo con todos los fragmentos: cabecera, menú, pie, `<head>` con Bootstrap y los mensajes flash.

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">

<!-- HEAD con parámetro: cada página pasa su título -->
<head th:fragment="head(titulo)">
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title th:text="${titulo} + ' | Club Numismático'">Club</title>
    <link
```

```
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css
" rel="stylesheet">
    <link th:href="@{/css/estilos.css}" rel="stylesheet">
</head>
<body>

<header th:fragment="cabecera" class="bg-dark text-white py-3">
    <div class="container"><h1 class="h3 mb-0">Club Numismático</h1></div>
<!-- CAMBIAR -->
</header>

<nav th:fragment="menu" class="navbar navbar-expand bg-body-tertiary mb-4">
    <div class="container">
        <ul class="navbar-nav">
            <li class="nav-item"><a class="nav-link"
th:href="@{/}">Inicio</a></li>
            <li class="nav-item"><a class="nav-link"
th:href="@{/monedas}">Monedas</a></li>
            <li class="nav-item"><a class="nav-link"
th:href="@{/colecciones}">Colecciones</a></li>
            <li class="nav-item"><a class="nav-link"
th:href="@{/coleccionistas}">Coleccionistas</a></li>
        </ul>
        <a class="btn btn-success btn-sm" th:href="@{/monedas/nueva}">+ Nueva
moneda</a>
    </div>
</nav>

<!-- MENSAJES FLASH (RedirectAttributes.addFlashAttribute) -->
<div th:fragment="mensajes">
    <div th:if="${mensaje}" class="alert alert-success" th:text="${mensaje}">
</div>
    <div th:if="${error}" class="alert alert-danger" th:text="${error}">
</div>
</div>

<footer th:fragment="pie" class="bg-dark text-white text-center py-3 mt-5">
    <small>&copy; 2026 Club Numismático · DWES</small>
    <script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.mi
n.js"></script>
</footer>

</body>
</html>
```

F.2 Esqueleto de CUALQUIER página (cópialo y rellena `<main>`)

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head th:replace="~{fragments/layout :: head('Monedas')}"></head>      <!--
CAMBIAR título -->
<body>
<header th:replace="~{fragments/layout :: cabecera}"></header>
<nav th:replace="~{fragments/layout :: menu}"></nav>
<main class="container">
    <div th:replace="~{fragments/layout :: mensajes}"></div>

    <!-- CONTENIDO DE LA PÁGINA -->

</main>
<footer th:replace="~{fragments/layout :: pie}"></footer>
</body>
</html>
```

Título dinámico: `head(${moneda.nombre})`.

F.3 Listado con filtro, búsqueda y borrar — `templates/monedas/lista.html` (dentro de `<main>`)

```
<h2>Monedas</h2>

<!-- Filtro + búsqueda: form GET → llegan como @RequestParam -->
<form class="row g-2 mb-4" th:action="@{/monedas}" method="get">
    <div class="col-md-4">
        <select class="form-select" name="coleccionId">
            <option value="">Todas las colecciones</option>
            <option th:each="c : ${colecciones}" th:value="${c.id}"
th:text="${c.nombre}"
                    th:selected="${c.id == coleccionId}"></option>
        </select>
    </div>
    <div class="col-md-5">
        <input class="form-control" type="text" name="nombre"
th:value="${nombre}" placeholder="Buscar por nombre">
    </div>
    <div class="col-md-3">
        <button class="btn btn-primary" type="submit">Filtrar</button>
        <a class="btn btn-outline-secondary"
th:href="@{/monedas}">Limpiar</a>
    </div>
</form>
```

```
<p th:if="${#lists.isEmpty(monedas)}" class="alert alert-info">No hay monedas
que coincidan.</p>

<table class="table table-striped align-middle"
th:unless="${#lists.isEmpty(monedas)}">
    <thead><tr><th>Imagen</th><th>Nombre</th><th>Año</th><th>Colección</th>
<th>Valor</th><th></th></tr></thead>
    <tbody>
    <tr th:each="m : ${monedas}">
        <td><img th:src="@{/imagenes/{img}(img=${m.imagen})}" width="60"
height="60" th:alt="${m.nombre}"></td>
        <td th:text="${m.nombre}">Nombre</td>
        <td th:text="${m.anio}">2000</td>
        <td th:text="${m.coleccion?.nombre}">Colección</td>
        <td th:text="${#numbers.formatDecimal(m.valorEstimado, 1, 'POINT', 2,
'COMMA')} + ' €'">0,00 €</td>
        <td class="text-end">
            <a class="btn btn-sm btn-primary" th:href="@{/monedas/{id}
(id=${m.id})}">Ver</a>
            <a class="btn btn-sm btn-warning"
th:href="@{/monedas/{id}/editar(id=${m.id})}">Editar</a>
            <form class="d-inline"
th:action="@{/monedas/{id}/borrar(id=${m.id})}" method="post"
                  onsubmit="return confirm('¿Seguro que quieres borrar esta
moneda?');">
                <button class="btn btn-sm btn-danger"
type="submit">Borrar</button>
            </form>
        </td>
    </tr>
    </tbody>
</table>
```

F.4 Detalle — `templates/monedas/detalle.html` (dentro de `<main>`)

```
<div class="row">
    <div class="col-md-5">
        <img class="img-fluid rounded border" th:src="@{/imagenes/{img}
(img=${moneda.imagen})}" th:alt="${moneda.nombre}">
    </div>
    <div class="col-md-7">
        <h2 th:text="${moneda.nombre}">Nombre</h2>
        <p th:text="${moneda.descripcion} ?: 'Sin descripción'"></p>
<!-- Elvis -->
        <p><strong>Año:</strong> <span th:text="${moneda.anio}"></span></p>
        <p><strong>Incorporada:</strong>
           <span th:text="${#temporals.format(moneda.fechaIncorporacion,
'dd/MM/yyyy')}"></span></p>
        <p><strong>Colección:</strong>
           <a th:href="@{/monedas(coleccionId=${moneda.coleccion.id})}"
th:text="${moneda.coleccion.nombre}"></a></p>
        <a class="btn btn-warning"
th:href="@{/monedas/{id}/editar(id=${moneda.id})}">Editar</a>
        <a class="btn btn-secondary" th:href="@{/monedas}">Volver</a>
    </div>
</div>
```

F.5 Formulario alta/edición con errores — `templates/monedas/formulario.html` (dentro de `<main>`)

```
<h2 th:text="${titulo}">Formulario</h2>

<!-- th:object = objeto del Model. enctype: OBLIGATORIO para subir ficheros -
->
<form th:action="@{/monedas/guardar}" th:object="${moneda}" method="post"
      enctype="multipart/form-data" novalidate>

    <!-- Todos los errores juntos -->
    <div class="alert alert-danger" th:if="${#fields.hasErrors('*')}">
        <ul class="mb-0"><li th:each="err : ${#fields.errors('*')}"
th:text="${err}"></li></ul>
    </div>

    <!-- Ocultos: id (vacío en alta, relleno en edición) e imagen actual -->
    <input type="hidden" th:field="*{id}">
    <input type="hidden" th:field="*{imagen}">

    <!-- TEXTO (repite este bloque por cada campo de texto: nombre, pais,
```

```
material...) -->
    <div class="mb-3">
        <label class="form-label" for="nombre">Nombre</label>
        <input class="form-control" type="text" th:field="*{nombre}"
th:errorclass="is-invalid">
        <div class="invalid-feedback" th:if="${#fields.hasErrors('nombre')}"
th:errors="*{nombre}"></div>
    </div>

    <!-- SELECT de un objeto relacionado: se envía coleccion.id -->
    <div class="mb-3">
        <label class="form-label" for="coleccion">Colección</label>
        <select class="form-select" id="coleccion" th:field="*{coleccion.id}"
                th:classappend="${#fields.hasErrors('coleccion')} ? 'is-
invalid'">
            <option value="">-- Elige una colección --</option>
            <option th:each="c : ${colecciones}" th:value="${c.id}"
th:text="${c.nombre}"></option>
        </select>
        <div class="invalid-feedback"
th:if="${#fields.hasErrors('coleccion')}" th:errors="*{coleccion}"></div>
    </div>

    <!-- TEXTAREA -->
    <textarea class="form-control" rows="3" th:field="*{descripcion}"
th:errorclass="is-invalid"></textarea>

    <!-- NÚMERO entero / decimal -->
    <input class="form-control" type="number" th:field="*{anio}"
th:errorclass="is-invalid">
    <input class="form-control" type="number" step="0.01" th:field="*
{valorEstimado}" th:errorclass="is-invalid">

    <!-- FECHA (el modelo necesita @DateTimeFormat(iso = ISO.DATE)) -->
    <input class="form-control" type="date" th:field="*{fechaIncorporacion}"
th:errorclass="is-invalid">

    <!-- FICHERO: name distinto de "imagen" -->
    <input class="form-control" type="file" id="archivoImagen"
name="archivoImagen" accept="image/*">
    <div class="text-danger small" th:if="${#fields.hasErrors('imagen')}"
th:errors="*{imagen}"></div>

    <button class="btn btn-success" type="submit">Guardar</button>
    <a class="btn btn-secondary" th:href="@{/monedas}">Cancelar</a>
</form>
```

- `novalidate` desactiva la validación del navegador, para que se vean los mensajes de Spring.
- `th:errorclass="is-invalid"` añade la clase de Bootstrap solo si ese campo tiene error; `invalid-feedback` se muestra entonces en rojo.
- Otros inputs: **checkbox** `<input type="checkbox" th:field="*{activo}">` (atributo `boolean`); **select de enum** `<option th:each="d : ${dificultades}" th:value="${d}" th:text="${d}">` con `@ModelAttribute("dificultades")` devolviendo `Dificultad.values()`.

F.6 Formatos útiles en plantillas

```
<span th:text="${#temporals.format(m.fecha, 'dd/MM/yyyy')}"></span>
<!-- fecha -->
<span th:text="${#numbers.formatDecimal(m.valor, 1, 'POINT', 2, 'COMMA')} + '
€'"></span> <!-- 1.234,50 € -->
<span th:text="${#lists.size(monedas)}"></span>
<!-- tamaño -->
<span th:text="${m.activo} ? 'Sí' : 'No'"></span>
<!-- ternario -->
<span th:text="${m.coleccion?.nombre}"></span>
<!-- null-safe -->
<tr th:each="m, st : ${monedas}"><td th:text="${st.count}"></td></tr>
<!-- nº de fila -->
```

G. Imágenes

Tres piezas: el formulario (F.5: `enctype` + `name="archivoImagen"`), la utilidad que guarda el fichero y la configuración que lo sirve. El controlador ya lo usa en el paso 5b de E.

G.1 `util/ImagenUtil.java` (sin anotaciones)

```
package es.daw.appcoleccion.util;

import org.springframework.web.multipart.MultipartFile;
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;
import java.util.UUID;

public class ImagenUtil {

    private ImagenUtil() { }
```

```
    // Guarda el fichero y devuelve el NOMBRE con el que se guardó (eso va en
moneda.imagen)
    public static String guardar(MultipartFile archivo, String carpeta)
throws IOException {
        String tipo = archivo.getContentType();
        if (tipo == null || !tipo.startsWith("image/")) {
            throw new IllegalArgumentException("El fichero debe ser una
imagen");
        }
        String original =
Paths.get(archivo.getOriginalFilename()).getFileName().toString();
        String nombre = UUID.randomUUID() + "_" + original;     // evita
pisar ficheros iguales

        Path ruta = Paths.get(carpeta).toAbsolutePath().resolve(nombre);
        Files.createDirectories(ruta.getParent());
        Files.write(ruta, archivo.getBytes());
        return nombre;
    }
}
```

G.2 `config/WebConfig.java` — servir la carpeta de subidas en `/imagenes/**`

```
package es.daw.appcoleccion.config;

import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.Configuration;
import
org.springframework.web.servlet.config.annotation.ResourceHandlerRegistry;
import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;
import java.nio.file.Paths;

@Configuration
public class WebConfig implements WebMvcConfigurer {

    @Value("${file.upload-dir}")
    private String uploadDir;

    @Override
    public void addResourceHandlers(ResourceHandlerRegistry registry) {
        String carpetaSubidas =
Paths.get(uploadDir).toAbsolutePath().toUri().toString();
        if (!carpetaSubidas.endsWith("/")) {
            carpetaSubidas += "/";                // la ubicación DEBE
acabar en "/"
```

```
        }
        registry.addResourceHandler("/imagenes/**")
                // 1º la carpeta de subidas; 2º static/imagenes (imágenes de
los datos iniciales)
                .addResourceLocations(carpetaSubidas,
"classpath:/static/imagenes/");
    }
}
```

Así **una sola URL** sirve tanto las imágenes iniciales (en `static/imagenes/`) como las subidas (en `uploads/`), y en las plantillas siempre se usa `th:src="@{/imagenes/{img}(img=${m.imagen})}"`. Pon también un `sin-imagen.png` en `static/imagenes/`.

G.3 Versión mínima de los apuntes (si vas justo de tiempo)

Sin `util` ni `config`, como en la UT4. Guarda en `uploads/` pero esas imágenes no se verán hasta que añadas `WebConfig`.

```
@PostMapping("/subir")
public String subirImagen(@RequestParam("archivoImagen") MultipartFile
archivo) {
    try {
        Path path = Paths.get(uploadDir, archivo.getOriginalFilename());
        Files.createDirectories(path.getParent());
        Files.write(path, archivo.getBytes());
        return "redirect:/ruta";
    } catch (IOException e) {
        return "redirect:/error";
    }
}
```

H. Persistencia con JPA (por si acaso)

El ejercicio de esta unidad es en memoria. Esto solo sirve si el examen pidiera base de datos. El controlador no cambia; cambian el modelo, el servicio y la configuración.

Dependencias: **Spring Data JPA** + **MySQL Driver** (o H2 Database para una BD en memoria sin instalar nada).

```
# application.properties
spring.datasource.url=jdbc:mysql://localhost:3306/club
spring.datasource.username=root
spring.datasource.password=
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
```

```
// model/Moneda.java
@Entity
@Data @NoArgsConstructor @AllArgsConstructor
public class Moneda {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String nombre;
    @ManyToOne
    @JoinColumn(name = "coleccion_id")
    private Coleccion coleccion;          // Coleccion también necesita
@Entity e @Id
}

// repository/MonedaRepository.java  (interfaz: Spring la implementa)
public interface MonedaRepository extends JpaRepository<Moneda, Long> {
    List<Moneda> findByNombreContainingIgnoreCase(String nombre);
    List<Moneda> findByColeccionId(Long coleccionId);
    @Query("SELECT m FROM Moneda m WHERE m.valorEstimado >= :min")
    List<Moneda> valiosas(@Param("min") Double min);
}

// service/MonedaServiceJpa.java
@Service
public class MonedaServiceJpa implements MonedaService {
    private final MonedaRepository repo;
    public MonedaServiceJpa(MonedaRepository repo) { this.repo = repo; }

    public List<Moneda> listarTodas()           { return repo.findAll(); }
    public Optional<Moneda> buscarPorId(Long id) { return repo.findById(id);
}
    public Moneda guardar(Moneda m)              { return repo.save(m); }
// alta o edición
    public boolean eliminar(Long id) {
        if (!repo.existsById(id)) return false;
        repo.deleteById(id);
```

```
        return true;
    }
    // ...
}
```

Si dejas a la vez `MonedaServiceImpl` y `MonedaServiceJpa` con `@Service`, la aplicación no arranca (dos beans del mismo tipo): quita la anotación a la que no uses.

Errores habituales y cómo solucionarlos

Síntoma	Causa probable	Solución
Whitelabel Error Page 404 en una URL	El controlador no está en un subpaquete del principal, falta `@Controller`, o la ruta no coincide	Revisa paquete, anotación y `@RequestMapping` + `@GetMapping`
La página muestra el texto `monedas/lista`	`@RestController` en lugar de `@Controller`	Cambia a `@Controller`
`TemplateInputException: Error resolving template [monedas/lista]`	La plantilla no existe en esa ruta o el nombre no coincide	`templates/monedas/lista.html`; el return sin `.html` ni `/` inicial
No arranca: `required a bean of type '...Service' that could not be found`	Falta `@Service` en la `Impl`	Añádela (en la implementación, no en la interfaz)
No arranca: `expected single matching bean but found 2`	Dos `@Service` implementan la misma interfaz	Deja solo una
No arranca: `Could not resolve placeholder 'file.upload-dir'`	La propiedad no está en `application.properties`	Añádela (sin comentarios al final de la línea)
No arranca: `Port 8080 was already in use`	Otra instancia sigue abierta	Para la anterior (■ en IntelliJ) o cambia `server.port`
`Neither BindingResult nor plain target object for bean name 'moneda'`	El GET del formulario no hizo `model.addAttribute("moneda", ...)`, o el nombre no coincide con `th:object`	Mismo nombre en el `addAttribute`, el `th:object` y el `@ModelAttribute("moneda")`
Al haber errores, el formulario peta porque falta `colecciones`	La lista del select solo se añadía en el GET	`@ModelAttribute("colecciones")` a nivel de método
La validación no hace nada	Falta `@Valid`, o falta la dependencia `spring-boot-starter-validation`	Añade ambas cosas
Error 400 al enviar un formulario con errores	`BindingResult` no está justo detrás del objeto validado	Orden: `@Valid @ModelAttribute X x, BindingResult r, ...`
Un número vacío pasa la validación	`@Min` / `@Max` aceptan null	Añade `@NotNull`
Error al convertir la fecha	Falta `@DateTimeFormat(iso = DateTimeFormat.ISO.DATE)`	Añádela en el atributo `LocalDate`
`UnsupportedOperationException` al añadir o borrar	La lista se creó con `List.of(...)` o con `.toList()`	Usa `new ArrayList<>()` para la lista que guardas
El mensaje de confirmación no aparece tras guardar	Usaste `model.addAttribute` antes de un `redirect:`	`redirectAttrs.addFlashAttribute` + fragmento `mensajes` en la página destino
Tras corregir errores se crea un duplicado en vez de editar	Falta el campo oculto `id` en el formulario	`<input type="hidden" th:field="*{id}">`

Síntoma	Causa probable	Solución
Al editar sin elegir imagen se pierde la anterior	Falta el campo oculto `imagen`	`<input type="hidden" th:field="*{imagen}">` + comprobar `archivo.isEmpty()`
El fichero llega vacío o `null`	Falta `enctype="multipart/form-data"` o el `name` no coincide con `@RequestParam`	Revisa ambos
La imagen subida no se ve (404)	`uploads/` está fuera de `static`	`WebConfig` (G.2)
`MaxUploadSizeExceededException`	Fichero mayor que el límite	Sube `max-file-size` y `max-request-size`
`NullPointerException` en la vista con `m.coleccion.nombre`	Alguna moneda tiene `coleccion` null	`${m.coleccion?.nombre}` y revisa los datos iniciales
`NoSuchElementException: No value present`	`.get()` sobre un Optional vacío	Comprueba `isEmpty()` antes
Getters en rojo, «cannot find symbol getNombre»	Lombok sin procesar	Plugin Lombok + Enable annotation processing
Cambios en HTML que no aparecen	Caché de Thymeleaf	`spring.thymeleaf.cache=false` y recarga (Ctrl+F5)
Todo el `th:` se ve tal cual en la página	Abriste el HTML como archivo, no desde el servidor	Entra por `http://localhost:8080/...`

Checklist antes de entregar

Requisitos de la Hoja04, para marcar uno a uno:

- [ ] Paquetes `model`, `service`, `controller`, `data`, `util`, `config`
- [ ] Tres clases de modelo: pieza (con objeto `Coleccion`), `Coleccion`, `Coleccionista`
- [ ] Pieza: nombre, descripción, año, valor estimado, fecha de incorporación, imagen + 2 atributos propios
- [ ] Coleccion: nombre y descripción · Coleccionista: nombre, email y fecha de alta
- [ ] Al menos una validación de obligatorio, una de rango numérico y una de fecha
- [ ] Cada servicio = interfaz + `Impl` en memoria; los controladores usan solo la interfaz
- [ ] Datos en `XxxData`, cargados en el constructor del servicio: 3 colecciones, 10 piezas, 4 coleccionistas
- [ ] Pieza: listado, detalle, alta, edición y borrado, con imagen
- [ ] Coleccion: listado · Coleccionista: listado y detalle
- [ ] Layout común con fragmentos (cabecera, menú, pie) y Bootstrap
- [ ] `@Valid` y errores visibles en el formulario
- [ ] Redirect con mensaje (`RedirectAttributes`) tras alta, edición y borrado
- [ ] Al menos 2 extras: filtro por colección, búsqueda por nombre, inicio con las últimas

- [ ] Prueba final: alta con errores, alta correcta con imagen, edición sin cambiar imagen, borrado, id inexistente (`/monedas/999`)